K

Krifmaz Kit

One shared foundation for every app: Encore's look, Encore's ease-of-use features, and the same fast, safe Rust and Kotlin building blocks — built once, used everywhere.

Encore · FSView · Filebrary · Photovault · Future apps

Document
Design & delivery plan

Target
kit v0.1.0

Repository
github.com/Krifmaz/kit

Date
November 2026

CONTENTS

What's inside

1. Why a kit 3
2. Principles 3
3. Architecture 4
4. Repository layout 5
5. Design tokens 6
6. Ease-of-use features 8
7. Contracts (shared APIs) 9
8. Server crates 9
9. kit-rules 10
10. Web: kit-leptos & Encore → Leptos 11
11. Android: kit-compose 12
12. Desktop: kit-iced 13
13. New-app template 13
14. Technology choices 14
15. Using the kit in an app 15
16. Adoption plan per app 16
17. Delivery: parallel tracks 17
18. Definition of done 18
19. Running hands-off 19
A. Coordinator prompt 20

> **In one sentence:** everything that makes Encore feel good — its look, its What's new popup, copyable errors, smart toasts, self-updating web app, in-app updates, bug reports, diagnostics, offline mode and settings forms — is pulled out of Encore into one shared repository, proven by switching Encore to it first, then reused by every other app.

SECTION 1–2

Why a kit

Encore was the first app and the place where the good ideas were worked out: the theme, the way errors can always be read in full and copied, the What's new popup after an update, the web app that quietly reloads into a new version, the banners that explain a disk problem in plain words. FSView, Filebrary and Photovault each need the same things. Without a shared base, every app re-implements them a little differently, fixes the same bugs again, and drifts apart in look and wording.

Today

- Each app copies or reinvents Encore's patterns.
- A fix in one app doesn't reach the others.
- Themes and wording drift apart.
- New apps start from scratch.

With the kit

- One implementation per feature, per platform.
- Fix once, every app gets it on the next kit release.
- One set of design tokens drives every theme.
- New apps start from a template with everything included.

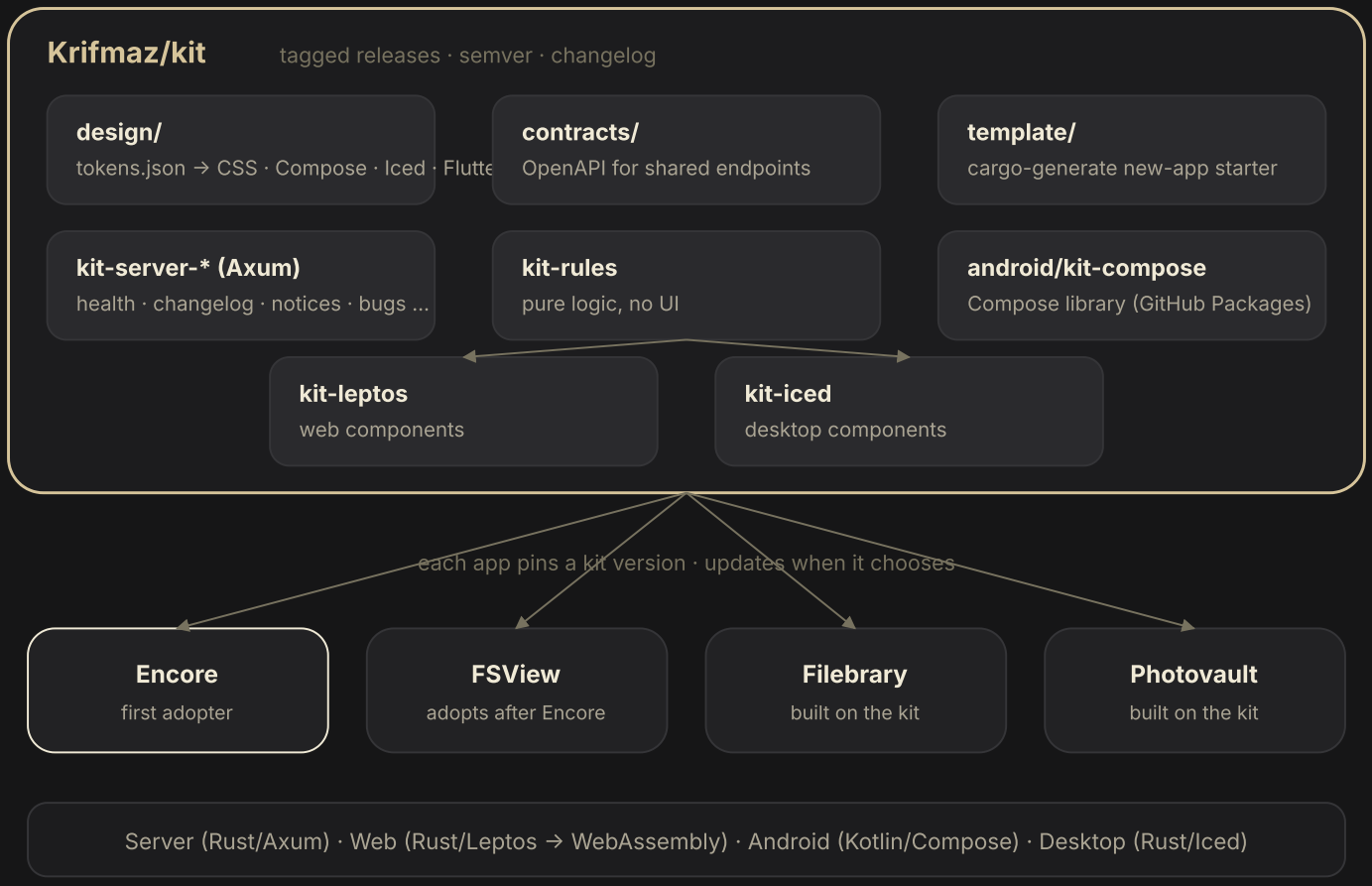
Principles

Principle	What it means in practice
Extract, don't invent	Every feature comes from Encore's working code. Behaviour and words stay exactly as Encore has them.
Encore first	Each piece is proven by switching Encore to it, with no visible change for Encore's users, before any other app adopts it.
Same words everywhere	A kit-wide parity test checks that every client (web, Android, desktop) shows the same strings — Encore's `parity.rs` rule, applied to all apps.
Tagged releases	Apps depend on a kit version (Cargo git tag, Gradle package). Each app chooses when to update, so one change can't break everything at once.
Fast and safe by default	Rust on servers and the web, Compose on Android, Iced on desktop; distroless non-root containers; no shell for user input; Baseline Profiles and benchmarks from day one.
Plain words, copyable errors	Every message says what happened and what to do; every error can be read in full and copied into a bug report.

SECTION 3

Architecture

One repository holds the shared pieces. Apps live in their own repositories and pull in tagged kit releases.



How the pieces connect

- **Tokens** feed every theme, so Encore's look is defined exactly once.
- **Contracts** pin down the shared endpoints, so every server speaks the same API to every client.
- **Server crates** implement the contracts; **kit-rules** holds the client logic once; **kit-leptos**, **kit-iced** and **kit-compose** draw it on each platform.
- The **template** wires all of it into a new app in one command.

SECTION 4

Repository layout

```
Krifmaz/kit
├── design/
│   ├── tokens.json            Encore's colours (all themes), fonts, sizes, radii, spacing, shadows, motion
│   └── gen/                   Rust generator → CSS variables · Compose theme · Iced theme · Flutter
ThemeData
├── contracts/                 OpenAPI + prose for every shared endpoint (section 7)
├── crates/
│   ├── kit-server-health      /api/health with version + web build hash
│   ├── kit-server-changelog   release notes, per-account "seen", popup switch
│   ├── kit-server-notices     plain-language banners with periodic + on-demand checks
│   ├── kit-server-bugreports  reports, attachments, crash logs, .txt export
│   ├── kit-server-diagnostics checks with status, message and fix
│   ├── kit-server-settings    schema → API → validation
│   ├── kit-server-updates     release feed (full release vs hotfix)
│   ├── kit-rules              pure logic: toasts, changelog, update detection, offline, uploads
│   ├── kit-leptos             web components (+ demo app)
│   └── kit-iced               desktop components
├── android/
│   └── kit-compose            Compose library: theme, toasts, dialogs, updater, bugs, offline, forms
├── template/                  cargo-generate starter for new apps
├── CHANGELOG.md
└── README.md
```

> **Why one repository?** The pieces change together (a new feature usually touches a contract, a server crate, kit-rules and three clients). One repo means one pull request, one version number and one changelog for the whole change — while apps still update on their own schedule by pinning a tag.

SECTION 5

Design tokens

Taken from Encore's stylesheet (`web-rs/static/css/app.css`, the `encore` theme: "the logo's colours: charcoal and cream") and its Compose theme. Encore's other themes (default, OLED black, light) are included too, so every app can offer the same theme picker.

Encore theme — colours

bg `#161617`	**bg-2** `#1c1c1e`	**panel** `#222224`	**panel-2** `#2a2a2d`
border `#353538`	**text / accent** `#f0ead6`	**accent-2** `#d6c49a`	**muted** `#aaa493`
faint `#77725f`	**ok** `#22c55e`	**warn** `#f59e0b`	**err** `#ef4444`

Typography

Use	Spec
Body	Inter 14px / 1.5
H1	28px, weight 750
H2	19px, weight 700
H3	15px, weight 650
Small	12px
Mono	ui-monospace stack

Shape & depth

Token	Value
radius	14px (cards), 10px (inputs, code)
shadow	0 10px 30px rgba(0,0,0,.35)
info	`#38bdf8`
on-accent	`#16161f`
themes	default · encore · oled · light

One source, four outputs

Output	Used by	Form
CSS variables	Leptos web apps, Encore web	`:root[data-theme="encore"] { --bg: … }`

Compose theme	Encore Android, Photovault Native	Kotlin `ColorScheme`, `Typography`, `Shapes` + Encore's `palette`
Iced theme	Encore desktop, future desktop apps	Rust `Theme` / palette and style functions
Flutter ThemeData	Photovault (Ente fork)	Dart `ThemeData` + colour extension

A test proves Encore's current CSS and Compose theme equal the generated output, so moving Encore onto the tokens changes nothing visually.

SECTION 6

Ease-of-use features

The features below exist in Encore today. The kit turns each into a shared building block on every platform.

Feature	What the user gets	From Encore
What's new & changelog	Release notes pop up once after an update; full changelog in Settings with a switch to turn the popup off. Hotfixes never re-trigger it.	`web-rs/src/whats_new.rs`, `/api/changelog/*`
Self-updating web app	An open tab notices the server updated and reloads into the new build by itself, without cutting off playback; waits out restarts, then reconnects.	`web-rs/src/update.rs`, service worker
In-app phone updates	Checks releases, downloads, verifies the signature and installs the APK from Settings.	`features/update/`
Notice banners	A plain-language banner when something's wrong ("the downloads folder is read-only — check the disk...") with Check again; re-checked automatically.	`web-rs/src/notice.rs`
Smart toasts	Error toasts stay until closed and have Copy; duplicates merge; Undo toasts; a cap on stacking; toasts show above dialogs.	`ui.rs`, `Toasts.kt`, `ToastRules.kt`
Read the whole error	Any cut-off text opens in full, selectable, with Copy — ready for a bug report.	`details.rs`, `Details.kt`
Bug reports & crash logs	In-app reports with screenshots and attachments; "Send crash log" files the phone's crash log for the admin.	`features/bugs/`
Diagnostics	One page that checks every connection, folder and tool, and says what's wrong and how to fix it.	Diagnostics (web + `DiagnosticsScreen.kt`)
Offline mode that explains itself	Stays usable from cached data, says why it's offline (no network, DNS, refused, timeout, HTTPS, proxy 502–504), queues actions to send later.	`features/offline/`
Drag & drop everywhere	Every upload accepts dropped files, with the same checks as the file picker.	`web-rs/src/dropzone.rs`
Settings from a schema	The server describes its settings; web and phone draw the forms, so new settings need no UI work.	`settings.toml`, `SchemaForm.kt`
Same words everywhere	A test that every interface uses the same wording.	`web-rs/src/parity.rs`

SECTION 7

Contracts (shared APIs)

Specs in `contracts/` match Encore's current API, so Encore's existing clients keep working while the server moves to the kit crates.

Endpoint / format	Purpose
`GET /api/health`	Server version and the web build hash; clients use it to detect updates and restarts.
`GET /api/changelog/new` `POST /api/changelog/seen`	Release notes newer than what this account has seen; mark a version seen. Popup on/off is a user preference.
Notices	Encore's storage notice generalised: id, severity, plain sentence, optional "check again" action; periodic and on-demand checks.
Bug reports	Create, list, view, status, attachments (images, .txt), crash-log kind, .txt export.
Diagnostics	A list of checks: name, status (ok / warn / fail), message, and how to fix it.
Settings schema	Settings described as data (type, label, help, default, validation, section), as in Encore's `server/crates/config/settings.toml`.
Update feed	The newest full release and its app assets; hotfix releases carry no apps and don't trigger app updates.

> **Rule:** a contract change is a kit release with a changelog entry. Servers and clients built against the same kit version always match.

SECTION 8

Server crates

Axum crates, one per feature, each added to an app in a few lines. Extracted from Encore's server; Encore's server switches to them with its tests still passing.

Crate	Provides
`kit-server-health`	Health endpoint with version and build hash.
`kit-server-changelog`	Notes, per-account seen tracking, popup switch, hotfix rule.
`kit-server-notices`	Register checks; periodic and on-demand re-check; the notices endpoint.
`kit-server-bugreports`	Reports, attachments, crash logs, .txt export.
`kit-server-diagnostics`	Check registry with status, message and fix.

`kit-server-settings`	Schema loading, settings API, validation.
`kit-server-updates`	Release feed for in-app updates.

SECTION 9

kit-rules

The logic Encore currently writes once per client, written once in plain Rust with no UI, and tested on its own.

Rules	Decides
Toasts	Errors stay with Copy; duplicates merge and move to newest; Undo; stack cap; how long each kind stays.
Changelog	Grouping notes into lists and tables; when the popup shows.
Update detection	When to reload, when to wait for the service worker, how to back off while the server restarts.
Offline	Classifying why the server can't be reached; the outbox of actions to send later, per account.
Uploads	Accept checks shared by the file picker and drag & drop.

SECTION 10

Web: kit-leptos & Encore → Leptos

Every web feature is built once as a Leptos component on top of kit-rules, using Encore's existing CSS classes so it looks identical. Encore's own web app moves to Leptos gradually, so there is one web stack and one component set for every app.

Components

- Toasts · details dialog with Copy
- What's new popup · changelog page
- Self-update reload (health + service worker)
- Notice banner with Check again
- Drop zones · schema settings form
- Bug report sheet · diagnostics page
- Offline banner
- Demo app showing every component

Why Leptos

- Same speed: still Rust → WebAssembly, fine-grained updates, no virtual DOM.
- Far less code than hand-written `web-sys`; no stale-UI bugs.
- Router, forms, server rendering and server functions built in.
- Plain CSS — Encore's stylesheet is reused unchanged.
- One component set instead of two.

Encore web migration (about 39,000 lines, 29 pages)

#	Step	Detail
1	Shell	Leptos takes over the app frame (navigation, player bar slot, toasts, dialogs) — the rest stays hand-written.
2	Settings	Schema form, diagnostics, What's new — proves the kit components inside Encore.
3	Remaining pages	Page by page; the most-used (Library, Player) once the pattern is proven.
4	Remove old code	When the last page moves, the hand-written DOM helpers go.

> **Guard rails:** same CSS, API calls and words on every page; `parity.rs` passes after every step; before/after screenshots and frame timings, with no speed regressions. Leptos is before 1.0, so its version is pinned and upgraded deliberately.

SECTION 11

Android: kit-compose

A Compose library extracted from Encore's Android app and published through GitHub Packages. Encore's app switches to it with no visible change and its unit tests passing.

Part	From Encore
Theme	Generated from the tokens (Encore's palette, typography, shapes).
Toasts & details	`Toasts.kt`, `ToastRules.kt`, `Details.kt`
What's new	Popup and changelog screen.
In-app updater	`features/update/`: release check, download, signature check, PackageInstaller.
Bug reports & crash logs	`features/bugs/`
Offline mode	`features/offline/`: banner, reasons, outbox.
Settings & diagnostics	`SchemaForm.kt`, `DiagnosticsScreen.kt`
Images	Encore's Coil setup: memory + disk cache, de-duplicated requests, preloading.
Performance setup	Baseline Profile + startup profile module, Macrobenchmark journeys, JankStats in debug.

Smoothness rules every app inherits

- Baseline Profile shipped in every release APK.
- Judge smoothness only on release builds (R8 on).
- Stable `key` and `contentType` on every lazy list.

- Fast-changing state (e.g. playback position) read only where it's shown, ideally in the draw phase.
- Images decoded at display size, preloaded ahead of the scroll.
- Disk, network and decoding never on the main thread (StrictMode in debug).

SECTION 12

Desktop: kit-iced

The same features as Iced components on top of kit-rules, styled from the tokens, extracted from Encore's desktop app (`clients/desktop`, Iced 0.14, native window drawn with wgpu — no browser engine), which then switches to them.

SECTION 13

New-app template

```
cargo generate krifmaz/kit template
```

You get	Details
Server	Axum with the kit server crates wired in (health, changelog, notices, bug reports, diagnostics, settings).
Web UI	Leptos with the kit components and the Encore theme.
Android (optional)	Compose app with kit-compose, Baseline Profile and benchmark modules.
Container	Distroless, non-root, read-only filesystem friendly, health check.
CI & releases	Calls the kit's shared workflows (below), so a new app ships a Docker image, web app, APK and desktop installers from day one.
`CLAUDE.md`	House rules: no shell for user input, full doc comments, plain wording, copyable errors, Encore's look, small validated changes.

Shared release pipeline (`.github/workflows/` in the kit)

The kit ships nothing itself. It holds Encore's release pipeline as reusable workflows; each app calls them in a few lines and ships its own Docker image, APK and installers.

Workflow	Does
`kit-ci.yml`	fmt, clippy, tests, web checks and parity on every PR.
`kit-release.yml`	On every push to `main`: next `vX.Y.Z` (PATCH by default, MINOR/MAJOR from `docs/releases/<version>.md`), normal release (never pre-release), release notes, SHA256SUMS.
`kit-docker`, `-android`, `-desktop`	Multi-arch GHCR image (`:<version>`, `:latest`); signed APK with Baseline Profile; Windows, Linux and macOS installers. All attached to the app's release.

SECTION 14

Technology choices

Layer	Choice	Why	Considered
Server	**Rust · Axum · Tokio**	Fast, small, safe; already used by Encore and FSView.	—
Database	**SQLite** (Postgres optional)	Zero extra containers for single-user and family apps.	Postgres-only
Web	**Leptos** (Rust → WASM)	Same speed as hand-written WASM, far less code, built-in router, forms, SSR.	Hand-written `web-sys`, Dioxus, React
Android	**Kotlin · Jetpack Compose**	Best for Android-only: native, light, smooth with the rules above.	Flutter (kept only for the Ente fork)
Desktop	**Iced**	Pure Rust, GPU-drawn, no browser engine; Encore already moved off Tauri.	Tauri, Electron, Slint
Shared logic on phones	**Rust core via UniFFI**	One tested implementation (crypto, sync, rules) for Android, desktop and server tests.	Re-implementing in Kotlin
Images (Android)	**Coil 3**	Caching, de-duplication and preloading proven in Encore.	Glide
Media (Android)	**Media3 / ExoPlayer**	Standard, used by Encore.	—
Releases	**cargo-dist** (or release-plz)	Same builds, installers and changelogs for every Rust app.	Custom workflows per repo
Plugins	**WebAssembly (Extism)** for new apps	Sandboxed, language-agnostic; Encore's Python SDK stays as is.	Native plugins

SECTION 15

Using the kit in an app

Rust (server, Leptos, Iced)

```
[dependencies]
kit-server-health    = { git = "https://github.com/Krifmaz/kit", tag = "v0.1.0" }
kit-server-changelog = { git = "https://github.com/Krifmaz/kit", tag = "v0.1.0" }
kit-leptos           = { git = "https://github.com/Krifmaz/kit", tag = "v0.1.0" }
```

Android (Gradle)

```
repositories { maven { url = uri("https://maven.pkg.github.com/Krifmaz/kit") } }
dependencies { implementation("dev.krifmaz.kit:kit-compose:0.1.0") }
```

Theme

```
// web:     import the generated tokens.css, set data-theme="encore" on <html>
// Android: KitTheme(theme = KitTheme.Encore) { App() }
// Iced:    kit_iced::theme::encore()
```

Updating

1. Read the kit's `CHANGELOG.md` for the new tag.
2. Bump the tag (Cargo) or version (Gradle) in one app.
3. Run that app's tests and look at its screens; then roll the same bump to the next app.

> **Package names** such as `dev.krifmaz.kit` are placeholders to settle in the contracts step; everything else in this section is the intended shape.

SECTION 16

Adoption plan per app

App	When	What changes
Encore	First	Server → kit crates; Android → kit-compose; desktop → kit-iced; web → Leptos (shell and Settings first, then page by page). No visible change for users.
FSView	After Encore	What's new popup and changelog, self-reloading web UI after updates, toasts with Copy, details dialog, notice banners for disk and pool problems, bug reports, diagnostics via the kit; web UI moves from React to Leptos page by page.
Filebrary	From the start	Built on the template: Leptos UI with kit components, kit server crates, Encore theme.
Photovault	From the start	Server uses the kit crates; Photovault Native (Compose) uses kit-compose; the Ente-fork app (Flutter) uses the generated Flutter theme.
New apps	Template	`cargo generate` and start building features, not plumbing.

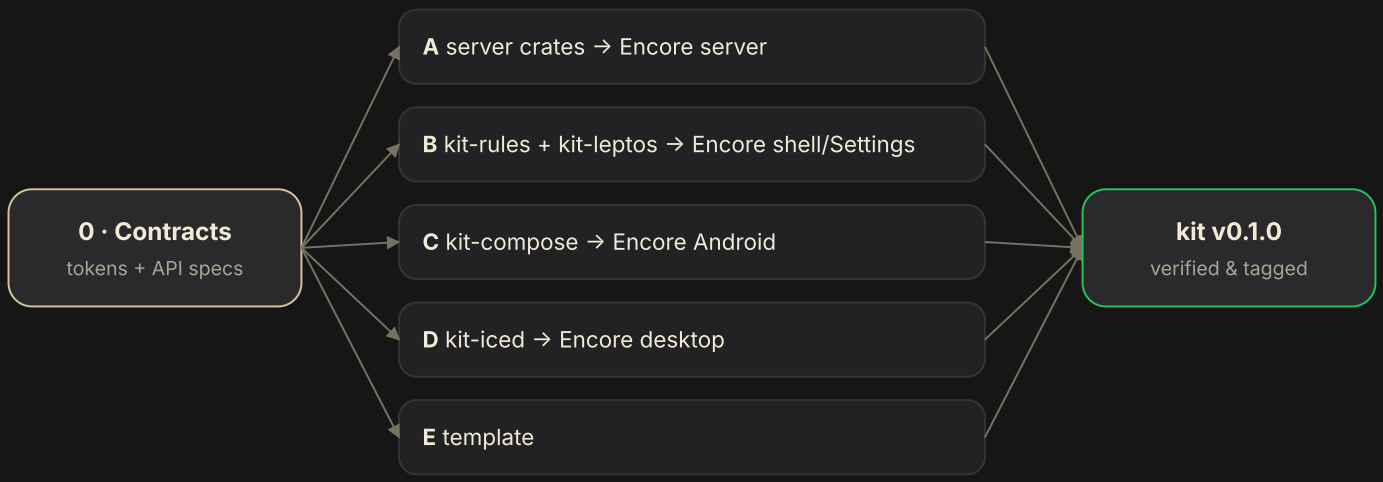
Feature coverage after adoption

Feature	Encore	FSView	Filebrary	Photovault
Encore theme	✓	✓	✓	✓
What's new & changelog	✓	✓	✓	✓
Self-updating web app	✓	✓	✓	✓
Smart toasts · copyable errors	✓	✓	✓	✓
Notice banners	✓	✓ disks & pools	✓ storage	✓ storage
Bug reports · crash logs	✓	✓	✓	✓
Diagnostics page	✓	✓	✓	✓
In-app APK updates	✓	—	if Android app	✓ Native
Offline mode	✓	—	✓ PWA	✓
Schema settings forms	✓	✓	✓	✓

SECTION 17

Delivery: parallel tracks

One short sequential step first, because everything else depends on it; then five tracks in parallel, each on its own branch and git worktree.



Merge order	Verify after each merge
1. Tokens & contracts	Token output equals Encore's current CSS and Compose theme.
2. Rules & server crates	Encore server tests; kit-rules tests.
3. Leptos, Compose, Iced	Web parity test, Android unit tests, all builds, before/after screenshots (web, Android, desktop), frame timings.
4. Template	A generated app builds, runs, and shows every kit feature.

> **Cost note:** parallel tracks finish sooner but use more tokens at once (roughly one agent's worth per track). The contracts step is what keeps the merges clean — don't skip it.

SECTION 18

Definition of done (v0.1.0)

	Criterion
☐	Tokens generate CSS, Compose, Iced and Flutter themes; Encore's current themes match exactly.
☐	Contracts for health, changelog, notices, bug reports, diagnostics, settings schema and update feed are published.
☐	All seven server crates exist; Encore's server runs on them with its tests passing.
☐	kit-rules covers toasts, changelog, update detection, offline and uploads, with tests.
☐	kit-leptos has every web component and a demo app; Encore's web shell and Settings run on Leptos, with parity passing.
☐	kit-compose is published; Encore's Android app runs on it with no visible change and its tests passing.
☐	kit-iced exists; Encore's desktop app runs on it.
☐	The template generates a working app with every kit feature.
☐	Every crate and module has a README and full doc comments; the kit changelog lists v0.1.0.
☐	Before/after screenshots show Encore looking and behaving exactly as before.

After v0.1.0

- Finish moving Encore's remaining web pages to Leptos, page by page.
- FSView adopts the kit and starts its Leptos move.
- Filebrary and Photovault build on the kit from here on.

SECTION 19

Running hands-off

The coordinator and every track work without check-ins. They decide, merge, tag and release on their own, and stop only for the few things listed under "Stop and ask".

Step	What the agent does on its own
Decisions	Makes the call itself, using this document, Encore's current behaviour and common sense, in that order. Writes each non-obvious choice as a one-line entry in `docs/DECISIONS.md` and keeps going.
Pull requests	Opens one PR per track and fixes CI and review-bot findings until green, then merges it (squash) and deletes the branch. GitHub does not let an account approve its own PR, so `main` requires green CI but no approving review.
Merge order	Merges in the order from Section 17. A track whose dependency is not merged yet rebases once it is, then merges.
Conflicts	Merges `main` into the branch, regenerates generated files with the repo's tooling, re-runs the tests and pushes.
Releases	Every merge to `main` releases automatically, the way Encore's hotfixes do, but as a normal release (never pre-release, rc, beta or edge) tagged `vMAJOR.MINOR.PATCH`. The release workflow bumps PATCH by default. The agent sets MINOR for new features and MAJOR for a breaking contract change by writing `docs/releases/<version>.md` in the PR, which also becomes the release notes. The first release is `v0.1.0`, once Section 18 passes.
App updates	Bumps Encore to the new kit tag in its own PR, merges it when green, and triggers Encore's normal release.
Failures	Finds the root cause and fixes it. Never skips, disables or deletes a test to get green, and never force-pushes `main`.
Report	Posts one summary at the end: what shipped, tags, PR links and the decisions log. No progress messages in between.

Stop and ask only when

- A change would delete user data or drop a database column that still has data.
- A secret, paid service or new account is needed.
- Encore would visibly change and this document doesn't say it should.

> **One-time repo setup (you, 2 minutes):** in `Krifmaz/kit` and each app repo, go to Settings → Branches → `main` rule: require status checks to pass and turn off "Require approvals". Under Settings → General, turn on "Allow auto-merge" and "Automatically delete head branches".

APPENDIX A

Coordinator prompt

Paste into the coordinator's "What do you want to work on?" box (under 8,000 characters). Give it access to Krifmaz/kit and Krifmaz/Encore.

```
Outcome: Krifmaz/kit, a shared foundation so all my apps (Encore, FSView, Filebrary, Photovault, future apps) share Encore's look and its ease-of-use features. Extract everything from Krifmaz/Encore: same behaviour, same wording, nothing invented. Prove each piece by switching Encore to it first, with no visible change for its users. MIT. Semver tags plus a changelog; apps depend on tagged releases (Cargo git tags; Android library via GitHub Packages).

KIT LAYOUT: design/ (tokens), contracts/ (API specs), crates/ (Rust), android/ (Compose library), template/ (new-app template).

1. DESIGN TOKENS: one tokens.json with Encore's exact colours (dark and light), fonts, sizes, radii, spacing, shadows and motion, taken from web-rs/static/css and the Android Compose theme. A Rust generator outputs CSS variables, a Compose theme, an Iced theme and Flutter ThemeData. A test proves Encore's current CSS and Compose theme match the tokens.

2. CONTRACTS: specs matching Encore's current API: /api/health (version, web build hash), /api/changelog/new and /api/changelog/seen, server notices (Encore's storage notice generalised: severity, sentence, check-again), bug reports with attachments and crash logs, diagnostics checks (name, status, message, fix), settings schema (server/crates/config/settings.toml), update feed (GitHub releases; full release vs hotfix rules).

3. SERVER CRATES (Axum), one per feature: health, changelog (per-account seen, popup switch, hotfixes don't re-trigger), notices (periodic and on-demand checks), bugreports (attachments, crash logs, .txt export), diagnostics, settings (schema, API, validation), updates. Extracted from Encore's server; Encore's server switches to them with its tests passing.

4. KIT-RULES: pure Rust logic with no UI: toast rules (errors stay with Copy, duplicates merged, Undo, stack cap), changelog grouping, update-detection rules, offline reasons (no network, DNS, refused, timeout, HTTPS, proxy 502–504) and outbox, upload checks.

5. KIT-LEPTOS AND ENCORE WEB → LEPTOS: build every web feature once as Leptos components on kit-rules, using Encore's existing CSS classes so they look identical: toasts, details dialog (full text with Copy), What's new and changelog, self-update reload (health polling plus service worker), notice banner with Check again, drop zones, schema settings form, bug report sheet, diagnostics, offline banner. Then move Encore's web-rs (about 39k lines, 29 pages) to Leptos gradually: shell first, then page by page (Settings, then Library, Player and the rest), with the rest hand-written until moved, so Encore keeps shipping. Same CSS, API calls and words; parity.rs passes after every page; before/after screenshots and frame timings with no regressions. Pin the Leptos version.

6. KIT-COMPOSE (Android library): extract from Encore's Android app: theme from tokens, Toasts/ToastRules, Details, What's new, in-app updater (features/update: GitHub check, download, signature check, PackageInstaller), bug reports and crash logs (features/bugs), offline banner, reasons and outbox (features/offline), SchemaForm, Diagnostics, Encore's Coil setup, Baseline Profile and Macrobenchmark setup. Encore's Android app switches to it with no visible change and its unit tests passing.

7. KIT-ICED: the same features as Iced components on kit-rules, extracted from clients/desktop, which switches to them.

8. TEMPLATE: cargo-generate template for new apps: Axum server with the kit crates, Leptos web UI with the kit components and the Encore theme, optional Compose Android app with kit-compose, distroless non-root Docker image, CI (fmt, clippy, tests, builds), cargo-dist releases, Baseline Profiles, and a CLAUDE.md with my conventions (no shell for user input, full doc comments, plain wording, copyable errors, Encore's look).
```

9. LATER (not now): FSView adopts the kit (What's new, self-reloading UI, toasts with Copy, details dialog, notice banners for disk problems, bug reports, diagnostics) and moves its React UI to Leptos; Filebrary and Photovault use the kit from the start.

HOW, IN PARALLEL: first a short sequential step: commit the tokens and contracts (everything else depends on them). Then parallel tracks on their own branches and worktrees: (a) server crates plus Encore server switch; (b) kit-rules plus kit-leptos plus Encore shell and Settings on Leptos; (c) kit-compose plus Encore Android switch; (d) kit-iced plus Encore desktop switch; (e) template; (f) shared release workflows (kit-ci, kit-release, kit-docker, kit-android, kit-desktop) taken from Encore's, with Encore switched to call them. Each track runs its own tests. Merge order: tokens and contracts → rules and server → leptos, compose and iced → template. Verify after every merge: Encore server tests, the web parity test, Android unit tests, all builds, and before/after screenshots of web, Android and desktop.

RULES: identical behaviour and words to Encore; a kit-wide parity test checks every client shows the same strings. Tests, clippy and fmt clean (ktlint for Kotlin), full doc comments, a README per crate and module, kit changelog entries.

DONE WHEN: kit v0.1.0 is tagged with tokens, contracts, server crates, kit-rules, kit-leptos, kit-compose, kit-iced and the template; Encore's server, Android app and desktop app run on the kit; Encore's web app runs on Leptos at least for the shell and Settings (remaining pages continue page by page); everything looks and behaves exactly as before.

AUTONOMY: work fully hands-off; do not wait for me. Make every judgement call yourself (this document first, then Encore's current behaviour, then common sense) and log non-obvious ones as one line each in docs/DECISIONS.md. For each track: open a PR, fix CI and bot findings until green, then squash-merge it yourself and delete the branch; merge in the Section 17 order, merging main into a branch when needed. Releases: no pre-releases, rc, beta or edge, ever. Add a release workflow that runs on every push to main and makes a normal GitHub release tagged vMAJOR.MINOR.PATCH (created through the release API), bumping PATCH by default; a PR raises MINOR (new feature) or MAJOR (breaking contract) by adding docs/releases/.md, which is also the release notes. The first release is v0.1.0 when the definition of done passes; every merge after that ships its own release. Bump Encore to each new kit tag in its own PR, merge it when green and run Encore's release. Never skip or disable tests, never force-push main. Stop and ask only for: deleting user data, a needed secret/paid service/new account, or a visible Encore change this document doesn't call for. Send me one summary at the end with tags, PR links and the decisions log.